APPLICATION NOTE

ATBM 驱动配置说明_FAQ

ATBM603X

1x1 802.11b/g/n Wi-Fi 芯片

Table of contents

1 单独编译驱动的编译方法 4

1.1 驱动通过 Make menuconfig 进行配置相关参数 4

(1) 进到驱动根目录执行，make menuconfig .....

1.2 配置说明 6

(1) 选择 wifi 芯片型号 .....

(2) 选择通信总线接口 .....

(3) 选择固件方式 .....

(4) 驱动一些扩展功能 .....

(5) 内部调试使用 .....

(6) 是否支持 WAPI .....

(7) 是否需要 short GI .....

(8) 修改 wifi 接口名称 .....

(9) 修改驱动名称以及挂载结点名称 .....

1.3 编译 15

(1) 修改驱动根目录的 Makefile .....

(2) 在同文件的 KERNELRELEASE 中添加内核路径以及编译器 .....

(3) 在同文件在底下，需要修改 PLATFORM_OTHER 的值 .....

(4) 编译 .....

2 驱动放置在内核中的编译方法 16

2.1 将驱动放置在内核中 16

3 驱动编译&加载出错 18

3.1 编译出错 18

3.2 加载出错 19

(1) NO_CONFIRM 宏没配置对导致出错 .....

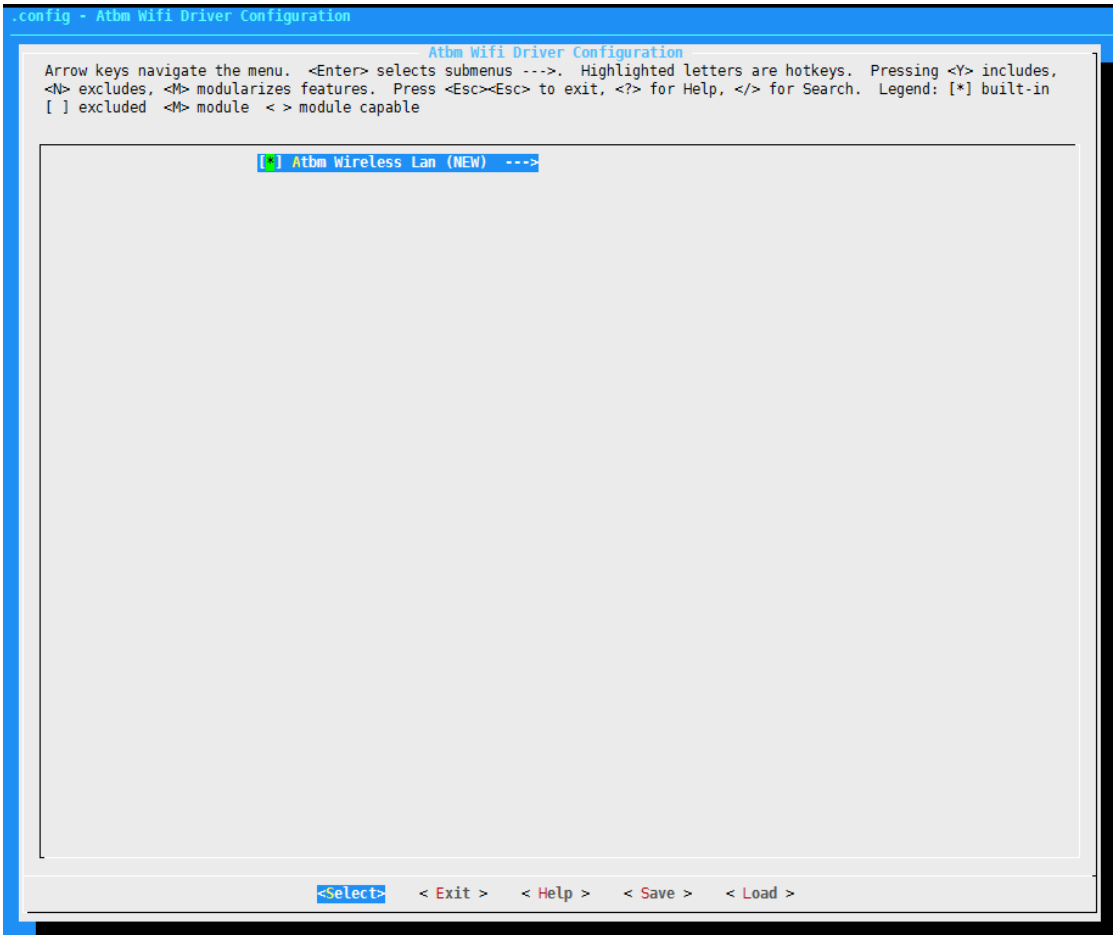
AN9310
Doc Rev: 1.2
Released:2020-06-10

作者	版本	说明
Yuzhihuang	V0.1	该文档适用于 SVN1359 版本以后的驱动
yuzhihuang	V0.2	增加调试等级信息说明
Yuzhihuang	V0.3	增加加载驱动固件方法说明
Yuzhihuang	V0.4	小改
Yuzhihuang	V0.5	小改
YUZHIHUAN	V1.1	正式版本
Yuzhihuang	V1.2	新增配置项，适用 1584 以后驱动

1 单独编译驱动的编译方法

1.1 驱动通过 Make menuconfig 进行配置相关参数

(1) 进到驱动根目录执行，make menuconfig



```
.config - Atbm Wifi Driver Configuration
                                Atbm Wireless Lan
Arrow keys navigate the menu.  <Enter> selects submenus --->.  Highlighted letters are hotkeys.  Pressing <Y> includes,
<N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in
[ ] excluded  <M> module  < > module capable

                        --- Atbm Wireless Lan
                        [*]   Atbm Wireless Lan (NEW)  --->

                        <Select>    < Exit >    < Help >    < Save >    < Load >
```

```
-*- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM603x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbmusbwifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x8888) Setting wifi usb pid
(atbm603x_wifi_usb) set module name
```

1.2 配置说明

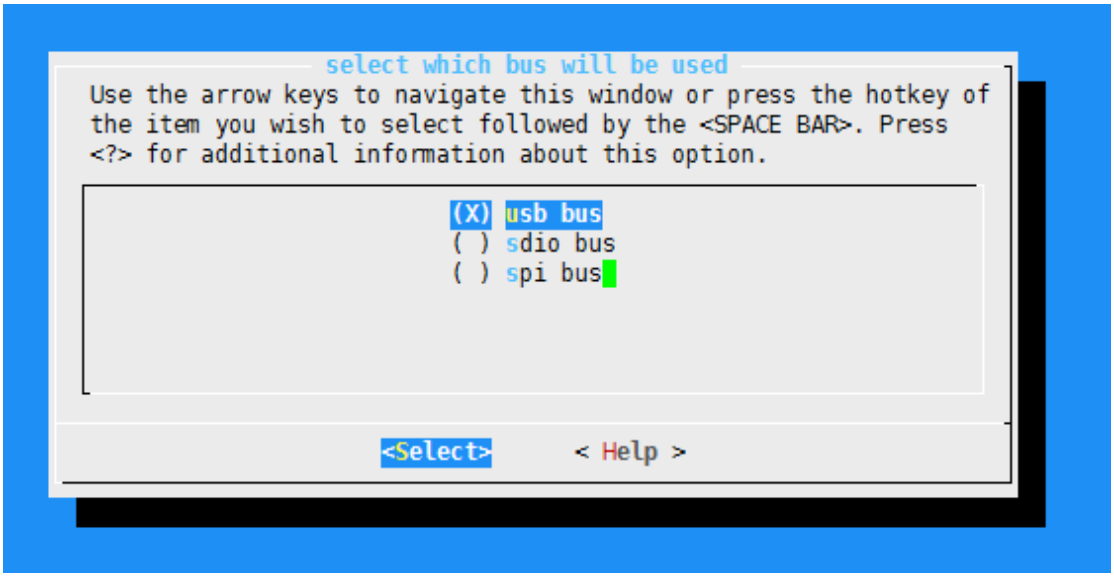
(1) 选择 wifi 芯片型号

```
-[-] Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM603x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support (NEW)
[*]   Use short GI support (NEW)
(wlan%d) Setting wifi interface 1 name (NEW)
(p2p%d) Setting wifi interface 2 name (NEW)
(atbm603x_wifi_usb) set module name (NEW)
```

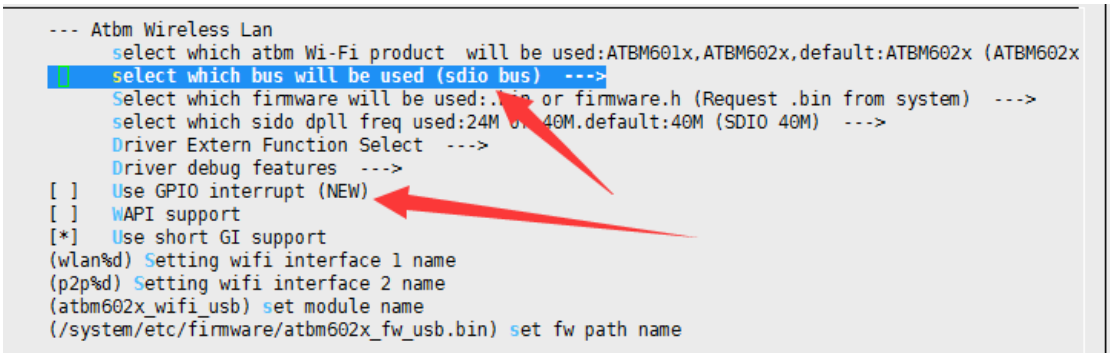
```
select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,defa
 Use the arrow keys to navigate this window or press the hotkey of
 the item you wish to select followed by the <SPACE BAR>. Press
 <?> for additional information about this option.

                    ( ) ATBM601x chip
                    ( ) ATBM602x chip
                    (X) ATBM603x chip

                <Select>     < Help >
```

(2) 选择通信总线接口

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM603x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support (NEW)
[*]   Use short GI support (NEW)
(wlan%d) Setting wifi interface 1 name (NEW)
(p2p%d) Setting wifi interface 2 name (NEW)
(atbm603x_wifi_usb) set module name (NEW)
```



如果选择 sdio 总线，需要选择是 sdio 中断还是 gpio 中断



a) 如果需要使用 GPIO 中断，

需要做如下修改，否则跳过此部分 ：

hal_apollo/Makefile 中，根据 platform 添加 GPIO 中断的宏。

```
ccflags-y += -DOPER_CLOCK_USE_SEM
ccflags-y += -DEXIT_MODULE_RESET_USB=0
ccflags-y += -DATBM_VIF_LIST_USE_RCU_LOCK
#ccflags-y += -DATBM_SUPPORT_SMARTCONFIG

ifeq ($(platform),PLATFORM_AMLOGIC_S805)
ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif
ifeq ($(platform),PLATFORM_FH_IPC)
ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif

ifeq ($(platform),PLATFORM_AMLOGIC)
#ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif
```

hal_apollo/apollo_plat.h 添加宏,该宏的值和顶层 Makefile 中 export 出来的值一样

```
*PLATFORM_FRIENDLY:based on linux3.086
*
*********************************************************
*/
#define PLATFORM_XUNWEI                         (1)
#define PLATFORM_SUN6I                          (2)
#define PLATFORM_FRIENDLY                       (3)
#define PLATFORM_SUN6I_64                       (4)
#define PLATFORM_CDLINUX                        (12)
#define PLATFORM_AMLOGIC_S805           (13)
#define PLATFORM_AMLOGIC_905            (8)
#define PLATFORM_FH_IPC                         (18)

#ifndef  ATBM_WIFI_PLATFORM
#define ATBM_WIFI_PLATFORM                      PLATFORM_XUNWEI
#endif
```

hal_apollo/atbm_platform.c 添加 GPIO 中断引脚号

```
^M
#endif  //CONFIG_ATBM_APOLLO_USE_GPIO_IRQ^M
#endif^M
struct atbm_platform_data platform_data = {^M
#if (ATBM_WIFI_PLATFORM == 10)^M
        .mmc_id       = "mmc1",^M
#else^M
        .mmc_id       = "mmc2",^M
#endif^M
        .clk_ctrl     = NULL,^M
        .power_ctrl   = atbm_power_ctrl,^M
        .insert_ctrl  = atbm_insert_crtl,^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_XUNWEI)^M
        .irq_gpio       = EXYNOS4_GPX2(4),^M
        .power_gpio     = EXYNOS4_GPC1(1),^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805)^M
        .irq_gpio       = INT_GPIO_4,^M
        .power_gpio     = 0,^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)^M
        .irq_gpio       = 100,^M
        .power_gpio     = 0,^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_FRIENDLY)^M
        .power_gpio     = EXYNOS4_GPK3(2),^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_FH_IPC)^M
        .irq_gpio       = 0,^M
#endif^M
        .reset_gpio = 0,^M
};^M
^M
struct atbm_platform_data *atbm_get_platform_data(void)^M
{^M
        return &platform_data;^M
```

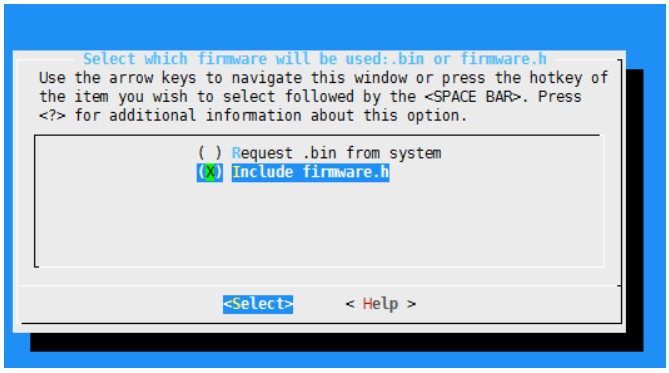
b) 整体设定完毕后，在驱动源码下执行 make 即可。

(3) 选择固件方式

可以选择独立于驱动之外的 bin 文件，或者选择包含在驱动里面的 hal_apollo/firmware.h

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM603x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support (NEW)
[*]   Use short GI support (NEW)
(wlan%d) Setting wifi interface 1 name (NEW)
(p2p%d) Setting wifi interface 2 name (NEW)
(atbm603x_wifi_usb) set module name (NEW)
```

IPC 考虑到内存的原因一般选择 firmware.h，安卓平台一般都是用的 bin 文件



出现下图的提示说明固件使用的是独立的 bin 文件，需要将红框固件名前缀去掉,只留下 **atbm602x_fw_usb.bin**。

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (sdio bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      select which sido dpll freq used:24M or 40M.default:40M (SDIO 40M)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   Use GPIO interrupt (NEW)
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```

PS:

1）固件使用 bin 文件需要内核支持下载 firmware,所以在内核需要打开 FW_LOADER 宏,不打开没法正常下载固件。

```
Symbol: FW_LOADER [=y]
Type  : tristate
Prompt: Userspace firmware loading support
  Location:
    -> Device Drivers
(1)   -> Generic Driver Options
  Defined at drivers/base/Kconfig:80
  Selected by: IXP4XX_NPE [=n] && ARCH_IXP4XX [=n] || PCM
```

```
.config - Linux/arm 4.4.192 Kernel Configuration
 Device Drivers  Generic Driver Options
                        Generic Driver Options
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.
  Pressing <Y> includes, <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.
  Legend: [*] built-in  [ ] excluded  <M> module  < > module capable

                        [*] Support for uevent helper
                        (/sbin/hotplug) path to uevent helper
                        [*] Maintain a devtmpfs filesystem to mount at /dev
                        [*]   Automount devtmpfs at /dev, after the kernel mounted the rootfs
                        [ ] Select only drivers that don't need compile-time external firmware
                        [ ] Prevent firmware from being built
                        <*> Userspace firmware loading support
                        [*]   Include in-kernel firmware blobs in kernel binary (NEW)
                        ()    External firmware blobs to build into the kernel binary (NEW)
                        [ ] Fallback user-helper invocation for firmware loading (NEW)
                        [ ] Allow device coredump
                        [ ] Driver Core verbose debug messages
```

2）最终的固件只能放在内核预定义的路径

2.1）该路径定义于：kernel/drivers/base/firmware_class.c

```
/* direct firmware loading support */
static char fw_path_para[256];
static const char * const fw_path[] = {
        fw_path_para,
        "/lib/firmware/updates/" UTS_RELEASE,
        "/lib/firmware/updates",
        "/lib/firmware/" UTS_RELEASE,
        "/lib/firmware"
};
```

2.2）在运行系统里面添加存放固件的路径

例如固件放的路径为/mnt/sdcard/firmware/添加路径的方法如下：

echo /mnt/sdcard/firmware/ > /sys/module/firmware_class/parameters/path

(4) 驱动一些扩展功能

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM603x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
[ ]   Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support (NEW)
[*]   Use short GI support (NEW)
(wlan%d) Setting wifi interface 1 name (NEW)
(p2p%d) Setting wifi interface 2 name (NEW)
(atbm603x_wifi_usb) set module name (NEW)
```

```
[*] Enable wifi interface bridge function
[*] Enable Tx no confirm function to enhance performance
[ ] Enable early suspend function for some platform power save
[*] Enable rx monitor function to receive all package
[ ]   Enable rx monitor header prism
[ ] Enable skb debug function to debug skb alloc and free
[ ] Enable memory debug function to debug memory alloc and free
[*] Enable 2.4g useing 5g channel function ,only support special frequnce
[ ] Enabel usb aggr tx funciton to enchance tx performance
[ ] Enable usb use dam buff for xmit
[ ] Enable usb cmd send directly function
[ ] Enable usb data send directly function
[ ] Enable usb wakeup reload fw function
[ ] Enable hw do tcp/ip checksum function
[ ] Enable P2P
[ ] enable sw enc function
[*] enabel dev_ctrl api
[*] enable modules fs
[ ] enable smartconfig function
[*] Enable loader driver fast function
[*] Enable iwpriv some prive func
```

a) Enable wifi interface bridge function
选择驱动是否支持桥接

b) Enable Tx no confirm function to enhance performance
这个功能默认是打开的。

c) Enable early suspend function for some platform power save
与平台相关，安卓系统层支持休眠时候需要打开。

d) Enable rx monitor function to receive all package
驱动是否支持进入监听状态的功能，默认打开
子选项：Enable rx monitor header prism
Monitor 头部修改为 PRISM 格式，默认为 RATIO 格式

e) Enable skb debug function to debug skb alloc and free
打开 skb 泄露 debug 的功能，此功能通常不打开

f) Enable memory debug function to debug memory alloc and free
打开 memory 泄露 debug 的功能，此功能通常不打开

g) Enable 2.4g useing 5g channel function ,only support special frequnce
是否使用 5G 信道作为特殊频点，默认关闭状态

h) Enabel usb aggr tx funciton to enchance tx performance
打开 usb 聚合发送数据包的功能，目前关闭，cpu 频率较低时可以打开，目前 sstar325 平台要打开此功能

i) Enable usb use dam buff for xmit
使用 usb 的 dma buff,一般打开 usb 聚合时打开此功能，目前 sstar325 平台打开此功能

j) Enable usb cmd send directly function
cpu 频率较低时打开此功能，目前 sstar325 平台需打开

k) Enable usb data send directly function
cpu 频率较低时打开此功能，目前 sstar325 平台打开此功能

l) Enable usb wakeup reload fw function
android 平台休眠唤醒时是否重新 load 固件，默认关闭

m) Enable hw do tcp/ip checksum function
是否使能 aresB 的 check sum 功能，默认关闭

n) Enable P2P
使能 P2P mirecast 功能

o) enable sw enc function
支持 enc 功能

p) enabel dev_ctrl api
使能 dev ioctl 相关命令功能

q) enable modules fs
使能 modules fs 功能，打开此功能会在/sys/module/<driver_name>/目录下生成 atbm_fs 目录该目录可以进行功能调试

r) enable smartconfig function
使能 smart config 功能

s) Enable loader driver fast function
打开此功能可以缩短加载驱动的时间

t) Enable iwpriv some prive func
打开此功能支持私有协议功能

(5) 内部调试使用

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```

(6) 是否支持 WAPI

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (sdio bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      select which sido dpll freq used:24M or 40M.default:40M (SDIO 40M)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   Use GPIO interrupt (NEW)
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```

WAPI 是我国首个在计算机宽带无线网络通信领域自主创新并拥有知识产权的安全接入技术标准。WAPI 同时也是中国无线局域网强制性标准中的安全机制。

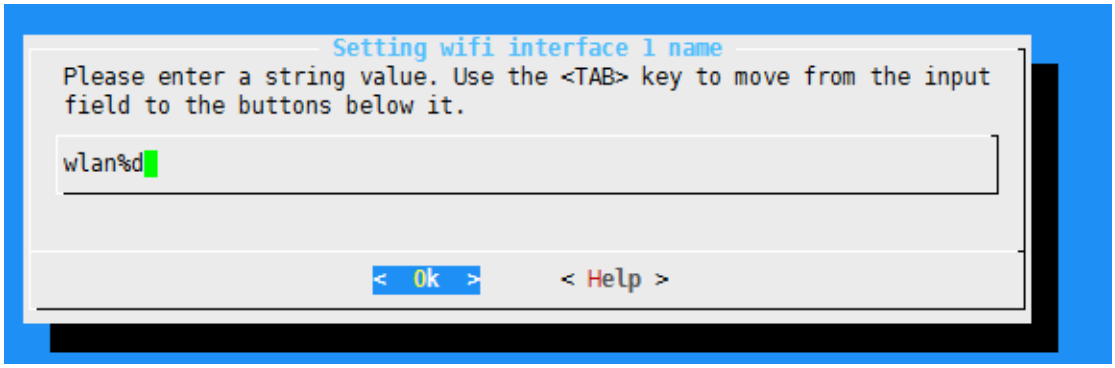
(7) 是否需要 short GI

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (sdio bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      select which sido dpll freq used:24M or 40M.default:40M (SDIO 40M)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   Use GPIO interrupt (NEW)
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```

该功能默认打开。

(8) 修改 wifi 接口名称

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (sdio bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      select which sido dpll freq used:24M or 40M.default:40M (SDIO 40M)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   Use GPIO interrupt (NEW)
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```



wifi 接口默认的名称是：wlan0 以及 p2p0.

客户可以根据需要执行修改：

1）客户想使用 wlan0 以及 wlan1,

可直接修改 p2p%d 修改为 wlan%d 即可。

2）客户想使用 wlan 以及 p2p 接口

将 wlan%d 修改为 wlan

将 p2p%d 修改为 p2p

(9) 修改驱动名称以及挂载结点名称

```
--- Atbm Wireless Lan
      select which atbm Wi-Fi product  will be used:ATBM601x,ATBM602x,default:ATBM602x (ATBM602x
      select which bus will be used (sdio bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      select which sido dpll freq used:24M or 40M.default:40M (SDIO 40M)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[ ]   Use GPIO interrupt (NEW)
[ ]   WAPI support
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
(p2p%d) Setting wifi interface 2 name
(atbm602x_wifi_usb) set module name
(/system/etc/firmware/atbm602x_fw_usb.bin) set fw path name
```

根据客户需要执行修改，编译出来生成的驱动名称。
以及在系统中挂载驱动 lsmod 看到的驱动名称。

1.3 编译

(1) 修改驱动根目录的 Makefile

PS：注意#PLATFORM_OTHER 20 ,这个值用户自定义，不要和已定义的冲突即可

```
#
#makefile for build atbm_wifi.ko
#
##################################################################
#
# when release to customer ,the CUSTOMER_SUPPORT_USED must set t
#
##################################################################
CUSTOMER_SUPPORT_USED=y
##################################################################
#PLATFORM_XUNWEI                1
#PLATFORM_SUN6I                          2
#PLATFORM_FRIENDLY                    3
#PLATFORM_SUN6I_64                    4
#PLATFORM_HI3798M                        5
#PLATFORM_HI3518E                        6
#PLATFORM_X86PC          7
#PLATFORM_AMLOGIC                        8
#PLATFORM_AMLOGIC_905X  9
#PLATFORM_ROCKCHIP      10
#PLATFORM_MSTAR                          11
#PLATFORM_CDLINUX                        12
#PLATFORM_AMLOGIC_S805  13
#PLATFORM_HIS_LINUX_3_4                  14
#PLATFORM_ROCKCHIP_3229                  15
#PLATFORM_ROCKCHIP_3229_ANDROID8         16
#PLATFORM_HS_IPC                         17
#PLATFORM_SIGMASTAR                    18
#PLATFORM_OTHER                          20
export
platform ?= PLATFORM_OTHER
#Android
#Linux
sys ?= Linux
#arch:arm or arm64
arch ?= arm
#export
#ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=$(platform)

ifeq ($(CUSTOMER_SUPPORT_USED),y)
MAKEFILE_SUB ?= Makefile.build.customer
else
MAKEFILE_SUB ?= Makefile.build
endif
```

客户使用

平台

平台系统及架构

(2) 在同文件的 KERNELRELEASE 中添加内核路径以及编译器

```
MAKEFILE_SUB ?= Makefile.build
endif

ifeq ($(KERNELRELEASE),)
ifeq ($(platform),PLATFORM_HS_IPC)
KERDIR:=/wifi_prj/staff/jilechang/HS_IPC_1115/kernel
CROSS_COMPILE:=/wifi_prj/staff/jilechang/HS_IPC_1115/opt/arm-anykav200-crosstool/usr/bin/arm-anykav200-linux-uclibcgnueabi-
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=17
arch = arm
endif

ifeq ($(platform),PLATFORM_OTHER)
KERDIR:=/wifi_prj/staff/jilechang/HS_IPC_1115/kernel
CROSS_COMPILE:=/wifi_prj/staff/jilechang/HS_IPC_1115/opt/arm-anykav200-crosstool/usr/bin/arm-anykav200-linux-uclibcgnueabi-
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=20
arch = arm
endif

ifeq ($(platform),PLATFORM_ROCKCHIP_3229_ANDROID8)
ifeq ($(sys),Android)
KERDIR:=/wifi_prj/staff/zhouzhanchao/rk3328_box_oreo_release_v1.0.0_20180319/kernel
CROSS_COMPILE:=/wifi_prj/staff/zhouzhanchao/rk3328_box_oreo_release_v1.0.0_20180319/prebuilts/gcc/linux-x86/aarch64/aarch64-linu
droid-4.9/bin/aarch64-linux-android-
else
#KERDIR:=/wifi_prj/staff/zhouzhanchao/iTop4412_Kernel_3.0/
```

(3) 在同文件在底下，需要修改 **PLATFORM_OTHER** 的值

```
ifeq ($(platform),PLATFORM_AMLOGIC_S805)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=13
endif
ifeq ($(platform),PLATFORM_ROCKCHIP_3229)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=10
endif
ifeq ($(platform),PLATFORM_SIGMASTAR)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=16
endif
ifeq ($(platform),PLATFORM_OTHER)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=20
endif
export
include $(src)/Makefile.build.kernel
endif
```

(4) 编译

编译有两种方式：

1. 只修改 3.1)&3.2)，然后在驱动根目录直接 make 即可
2. 只修改 3.1&3.3，然后在驱动根目录执行
   make CROSS_COMPILE=交叉编译器 KERDIR=内核路径

2 驱动放置在内核中的编译方法

2.1 将驱动放置在内核中

进入内核目录下的 drivers/net/wireless/子目录修改 Makefile 和 Kconfig 文件

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
fugui@ubuntu200: /wifi_prj/staff/fugui/platform/iTop4412_Kernel_3.0$ cd drivers/net/wireless/
fugui@ubuntu200: /wifi_prj/staff/fugui/platform/iTop4412_Kernel_3.0/drivers/net/wireless$
```

a) 修改 Makefile

```
obj-$(CONFIG_HOSTAP)            += hostap/
obj-$(CONFIG_B43)               += b43/
obj-$(CONFIG_B43LEGACY)         += b43legacy/
obj-$(CONFIG_ZD1211RW)          += zd1211rw/
obj-$(CONFIG_RTL8180)           += rtl818x/
obj-$(CONFIG_RTL8187)           += rtl818x/
obj-$(CONFIG_RTLWIFI)           += rtlwifi/
obj-$(CONFIG_ATBM_WIRELESS)     += atbm_wifi_40M/

# 16-bit wireless PCMCIA client drivers
obj-$(CONFIG_PCMCIA_RAYCS)      += ray_cs.o
obj-$(CONFIG_PCMCIA_WL3501)     += wl3501_cs.o
```

b) 修改 Kconfig

```
source "drivers/net/wireless/rt2x00/Kconfig"
source "drivers/net/wireless/rtlwifi/Kconfig"
source "drivers/net/wireless/wl1251/Kconfig"
source "drivers/net/wireless/wl12xx/Kconfig"
source "drivers/net/wireless/zd1211rw/Kconfig"
source "drivers/net/wireless/mwifiex/Kconfig"
source "drivers/net/wireless/atbm_wifi_40M/Kconfig"

menuconfig MTK_WIRELESS_SOLUTION
        bool "MTK wireless chip configuration"
        help
          "enable/disable and config MTK wireless solution"

if MTK_WIRELESS_SOLUTION
source "drivers/net/wireless/combo_mt66xx/Kconfig"
source "drivers/net/wireless/mt5931/Kconfig"
endif # MTK_WIRELESS_SOLUTION
endif # WLAN
```

修改完 Makefile 和 Kconfig 文件后回到内核顶层目录

将 atbm_wifi_40M 的驱动源码目录复制到内核目录下的 drivers/net/wirelesee/

c) 修改 atbm_wifi_40M 中的 Makefile,指定相关平台选择，默认指定为 platform_other,

PS：注意#PLATFORM_OTHER 20，这个值用户自定义，不要和已定义的冲突即可

```
Makefile
28 #PLATFORM_OTHER                20
29 export
30 platform ?= PLATFORM_OTHER      选择平台
31 #Android
32 #Linux
33 sys ?= Linux
34 #arch:arm or arm64              选择系统及架构
35 arch ?= arm
36 #export
37 #ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=$(platform)
38
39 ifeq ($(CUSTOMER_SUPPORT_USED),y)
40 MAKEFILE_SUB ?= Makefile.build.customer
41 else
42 MAKEFILE_SUB ?= Makefile.build
43 endif
44
45 ifeq ($(KERNELRELEASE),)
46
47 ifeq ($(platform),PLATFORM_HS_IPC)
48 KERDIR:=/wifi_prj/staff/zhouzhanchao/ankai_hs_ipc/kernel/kernel_testXMFlash/kernel/
49 CROSS_COMPILE:=/wifi_prj/staff/zhouzhanchao/ankai_hs_ipc/bin/arm-2009q3/bin/arm-none-linux-gnueabi-
50 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=17
51 arch = arm
```

在同文件在底下，需要修改 PLATFORM_OTHER 的值

```
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=12
endif
ifeq ($(platform),PLATFORM_AMLOGIC_S805)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=13
endif
ifeq ($(platform),PLATFORM_ROCKCHIP_3229)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=10
endif
ifeq ($(platform),PLATFORM_XIONGMAI)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=21
endif
ifeq ($(platform),PLATFORM_OTHER)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=20
endif
export
include $(grc)/Makefile.build.kernel
endif
```

d) 通过 make menuconfig 配置 atbm_wifi 驱动支持的相关配置

➔ 请参考文档的【一.2)】：配置驱动。

e) 通过平台相关编译方式编译得到 atbm 的驱动 ko 文件。

f) 编译出来的驱动有点大 需要 strip 缩小体积

Arm-linux-xxx-strip --strip-debug atbm_wifixxx.ko

3 驱动编译&加载出错

3.1 编译出错

在编译驱动时，有可能出现编译限制等级较为严格导致出错。

```
  ^
  CC [M]  drivers/net/wireless/atbm_HS_svn950/hal_apollo/pm.o
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:90:9: note: #pragma message: xunwei
 #pragma message(PLATFORMINF)
         ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: implicit declaration of function 'EXYNOS4_GPX2' [-Werror=implicit-function-declaration]
  .irq_gpio = EXYNOS4_GPX2(4),
  ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: initializer element is not constant
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: (near initialization for 'platform_data.irq_gpio')
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: implicit declaration of function 'EXYNOS4_GPC1' [-Werror=implicit-function-declaration]
  .power_gpio = EXYNOS4_GPC1(1),
  ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: initializer element is not constant
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: (near initialization for 'platform_data.power_gpio')
cc1: some warnings being treated as errors
make[5]: *** [drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.o] Error 1
make[5]: *** Waiting for unfinished jobs....
drivers/net/wireless/atbm_HS_svn950/hal_apollo/hwio_usb.c: In function 'atbm_before_load_firmware':
```

修改 kernel/Makefile

```
KBUILD_CFLAGS   += $(call cc-option,-Wno-stringop-truncation)
KBUILD_CFLAGS   += $(call cc-option,-Wno-sizeof-pointer-memaccess)
KBUILD_CFLAGS   += $(call cc-option,-Wno-array-bounds)
KBUILD_CFLAGS   += $(call cc-option,-Wno-packed-not-aligned)

# disable warnings in gcc 7.2.1
#KBUILD_CFLAGS   += $(call cc-option,-Wno-switch-unreachable)
#KBUILD_CFLAGS   += $(call cc-option,-Wno-misleading-indentation)

# use the deterministic mode of AR if available
KBUILD_ARFLAGS := $(call ar-option,D)

#KBUILD_CFLAGS   += -Werror

include scripts/Makefile.kasan
include scripts/Makefile.extrawarn
include scripts/Makefile.ubsan

# Add any arch overrides and user supplied CPPFLAGS, AFLAGS and CFLAGS as
# last assignments
KBUILD_CPPFLAGS += $(ARCH_CPPFLAGS) $(KCPPFLAGS)
KBUILD_AFLAGS   += $(ARCH_AFLAGS)   $(KAFLAGS)
KBUILD_CFLAGS   += $(ARCH_CFLAGS)   $(KCFLAGS)

# Use --build-id when available.
LDFLAGS_BUILD_ID = $(patsubst -Wl$(comma)%,%,\
                   $(call cc-ldoption, -Wl$(comma)--build-id,)
KBUILD_LDFLAGS_MODULE += $(LDFLAGS_BUILD_ID)
```

如果注释上面的宏还不行的话，就需要按照下面的一个个修改。

```
                    -include $(srctree)/include/linux/kconfig.h

KBUILD_CPPFLAGS := -D__KERNEL__

KBUILD_CFLAGS   := -Wall -Wundef -Wstrict-prototypes -Wno-trigraphs \
                   -fno-strict-aliasing -fno-common \
                   -Wno-format-security \
                   -fno-delete-null-pointer-checks
#-Werror-implicit-function-declaration
KBUILD_AFLAGS_KERNEL :=
KBUILD_CFLAGS_KERNEL :=
KBUILD_AFLAGS   := -D__ASSEMBLY
```

类似警告导致 error 的问题，类似修改。

3.2 加载出错

(1) NO_CONFIRM 宏没配置对导致出错

```
:CAPABILITIES_SHARE_CRYSTAL        [0]
:CAPABILITIES_HW_CHECKSUM           [0]
:CAPABILITIES_SINGLE_CHANNEL_MULRX [1]
:CAPABILITIES_CFO_DCXO_CORRECTION  [0]
:LMAC SET CAPABILITIES_NO_CONFIRM <ERROR>
-[ cut here ]------------
at bfd62b14 [verbose debug info unavailable]
ror: Oops - BUG: 0 [#1] PREEMPT SMP THUMB2
```

解决办法需要在一开始配置驱动时候打开对应的宏，如果打开了就给关闭。

```
.config - Atbm Wifi Driver Configuration
                    Driver Extern Function Select
  Arrow keys navigate the menu.  <Enter> selects submenus --->.  Highlighted letters are hotkeys.  Pres
  <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Leg
  [ ] excluded  <M> module  < > module capable

                    [*] Enable wifi interface bridge function
                    [*] Enable Tx no confirm function to enhance performance
                    [ ] Enable early suspend function for some platform power save
```

CONTACT INFORMATION

AltoBeam (China) Inc.

Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083

Tel: (8610) 6270 1811

Fax: (8610) 6270 1830

Website: www.altobeam.com

Email: support@altobeam.com

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2020 AltoBeam, all rights reserved